MDERank: A Masked Document Embedding Rank Approach for Unsupervised Keyphrase Extraction

Linhan Zhang[12*] **Qian Chen**[2] **Wen Wang**[2] **Chong Deng**[2] **Shiliang Zhang**[2]
Bing Li[3] **Wei Wang**[4] **Xin Cao**[1]
[1]School of Computer Science and Engineering, The University of New South Wales
[2]Speech Lab, Alibaba Group, China
[3]Institute of High Performance Computing (IHPC), Agency for Science, Technology & Research (A*STAR), Singapore
[4]Hong Kong University of Science and Technology (Guangzhou), China
{linahan.zhang, xin.cao}@unsw.edu.au
{tanqing.cq,w.wang,dengchong.d,sly.zsl}@alibaba-inc.com
li_bing@ihpc.a-star.edu.sg weiwcs@ust.hk

arXiv:2110.06651v1 [cs.CL] 13 Oct 2021

Abstract

Keyphrases are phrases in a document providing a concise summary of core content, helping readers to understand what the article is talking about in a minute. However, existing unsupervised works are not robust enough to handle various types of documents owing to the mismatch of sequence length for comparison. In this paper, we propose a novel unsupervised keyword extraction method by leveraging the BERT-based model to select and rank candidate keyphrases with a MASK strategy. In addition, we further enhance the model, denoted as Keyphrases Extraction BERT (KPEBERT), via designing a compatible self-supervised task and conducting a contrast learning. We conducted extensive experimental evaluation to demonstrate the superiority and robustness of the proposed method as well as the effectiveness of KPEBERT.

1 Introduction

With the highly development of representation learning, BERT (Devlin et al., 2018) based embeddings achieved encouraging performance in all Neural Language Processing (NLP) tasks (Liu and Lapata, 2019; Sun et al., 2019; Jia et al., 2020). As for keyphrase extraction task (KPE), there have been numerous works attempting to utilizing BERT and its variants (Liu et al., 2019; Yang et al., 2019; Beltagy et al., 2020; Clark et al., 2020; Zaheer et al., 2020) to encode documents. The main limitation of previous BERT-based works, however, is either treating keyphrases extraction as sequence tagging problem or taking BERT as a generative model in a supervised way. It means that they need large-scaled annotated data. Nevertheless, the quantity of annotated data is hard to realize large-scale pretraining. Instead, currently industry prefers to deploying unsupervised KPE methods such as TextRank (Mihalcea and Tarau, 2004), YAKE (Campos et al., 2018), EmbedRank (Bennani-Smires et al., 2018) etc.

Indeed, traditional unsupervised KPE methods could ensure high retrieval speed and outstanding performance on certain datasets; however, it could not be ignored that they are usually not robust on different datasets with inductive bias. This is because that above methods are based on features designed by human consciously. Hence, the selection of data features has a significant impact on the final experimental results.

Besides, there are few works (Bennani-Smires et al., 2018; Sun et al., 2020) using document embeddings to conduct unsupervised keyphrase extraction. They follow the pipeline: (1) select candidates from a document firstly according to POS tags and stopwords, deemed as rules filters. (2) Rank candidates based on scoring function. Most of these embedding-based methods are relying on statistic-based embedding model like Sent2Vec (Pagliardini et al., 2017) or language model weaker than BERT like ELMo (Peters et al., 2018). At the same time, embedding-based methods compare the similarity score of source document and candidates embeddings, and then rank candidates by its score in a descending order. Top-K candidates are chosen as keyphrases. In this paper, we call above methods as Phrase-Document based methods, where the frequency and sequence length of a candidate play an important role in the semantics of a document. The is because the input sequence length of a document and a phrase is unequal, inducing hard to aligning in semantic space. A longer phrase may get a higher score,

*Work is done during the internship at Speech Lab, Alibaba Group.

Document: contractor selection for a project is an important decision, one for the project time and cost, next for the quality obtained by the project. although the project managers can easily determine the project time and cost, the quality is usually undefined especially for un-experienced managers. with a learnable property, an approach is first introduced in this paper to quantify the quality obtained for a gas well drilling project. then, based on these three objectives (time, cost, and quality), a contractor selection problem is converted to an optimization problem. next, the nsga-ii algorithm is utilized for solution. at the end, a sensitivity analysis is performed to select the parameters of the algorithm.

Gold Labels:
contractor selection, contractor selection problem, cost, determine the project time and cost, gas well drilling project, learnable property, NSGA-II algorithm, objectives, optimization problem, project managers, project time and cost, quality, quantify the quality obtained for a gas well drilling project, sensitivity analysis, time, un-experienced managers

Baseline Top-15 prediction:
nsga-ii algorithm, un-experienced managers, contractor selection problem, learnable property, sensitivity analysis, project managers, contractor selection, project time, optimization problem, important decision, parameters, quality, objectives, project, algorithm

MDERank Top-15 prediction:
project time, un-experienced managers, contractor selection problem, nsga-ii algorithm, project, quality, cost, learnable property, sensitivity analysis, time, important decision, optimization problem, project managers, contractor selection, paper

Figure 1: The example document is underlined and the orange line represents a overlapping keyphrase. Baseline method is Phrase-Document, the method adopted the pipeline of embedding-based works but replacing the encoder with a BERT model. The results for baseline matched with gold labels are marked in red, while the results for MDERank are marked in green.

which is unfair for short phrases. In this scenario, such methods suffer from a strong bias so that they prefer long and high frequency candidates. Additionally, the embedding for candidates obtained from BERT is unreliable owing to the lack of context. Figure 1 illustrates a case for this issue, where Phrase-Document baseline method has a preference on longer candidates.

Considering the insufficient annotated data and the capability of BERT modeling, we present a novel embedding-based keyphrase extraction method based on document embedding, denoted as masked document embedding rank (MDERank). The framework of MDERank is shown in Figure 2. It is intuitive that if a document without one or more keyphrases, then its semantic would be different from its source. Therefore, instead of comparing document with its candidate directly, for a document, we mask one of its candidates and get a masked document. Then, the similarity score of masked document and its source document will be the final ranking score. Opposite with embedding-based KPE method, a higher similarity score in MDERank means a lower ranking for a candidate, which is in consist with the basic idea it will be dissimilar to its source document if a document masked with its keyphrase. In this way, we could make sure the alignment of input length for similarity comparison and eliminate the effect of frequency and sequence length by designing variants for masking strategy flexibly. We advance four types of masking strategies: mask all, mask once, mask the highest and mask subset to control the number of MASK placeholders. Specially, mask subset could help to improve the diversity of predicted keyphrases because it address the overlapping issue by allowing each position to be only masked once.

Since pre-trained BERT not adapted to KPE scenario, we propose a self-supervised method to fine-tune BERT model (KPEBERT). It is essential for BERT to tell difference between keyphrases and non-keyphrases, so we apply unsupervised KPE methods as YAKE for producing keyphrases as pseudo labels and train BERT by a contrastive learning framework. Through absolute and relative sampling, we could control the confidence of previous KPE method for our model training. Absolute denotes the keyphrases and non-keyphrases directly assgined by unsupervised KPE results. As for relative sampling, it exploits ranking in unsupervised KPE result to decide which candidate is a keyphrase: a higher rank is a keyphrase compared to the one with a lower rank. Thus, the number of hard example is increased to make our model learn to rank high-quality candidates.

We conduct comprehensive evaluations of MDERank and KPEBERT model on six standard keyphrases extraction datasets. Our MDERank achieves 17.0%, 21.99% and 23.85% for average F1@5, F1@10, F1@15 respectively, a 1.71%, 1.46% and 1.86% absolute improvement compared to state-of-the-art results. The fine-tuned BERT model further promotes the MDERank method and improves F1 score by 1.70%, 2.18% and 1.74%. Additionally, we conduct experiments for variants of mask strategies to prove the validity of our basic ideas. MDERank shows prominent performance on long text datasets and we give explanation via experiments, which analyzed the impact of sequence length.

2 Related Work

This section briefly describes the related work from two aspects: unsupervised keyphrases extraction

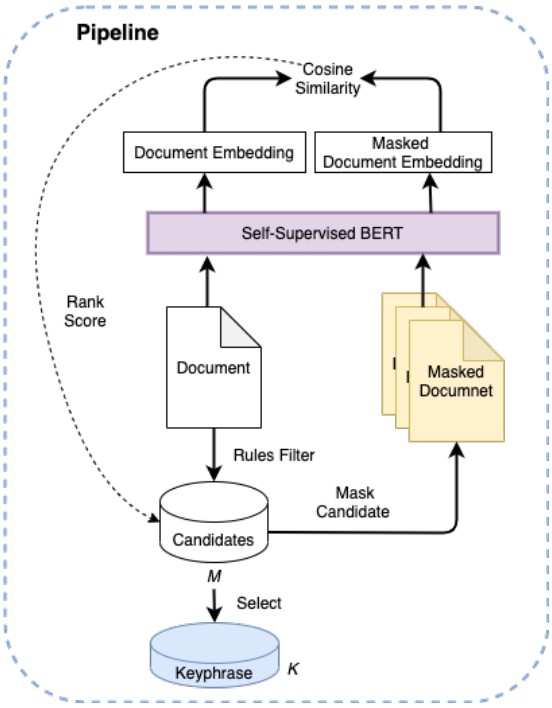

Figure 2: Blue-dotted border illustrates the pipeline of our work.

and contextual embeddings.

2.1 Unsupervised Keyphrases Extraction

Keyphrases extraction is a common problem in NLP area. In the past few years, researchers have developed a wide range of solutions for this task, including supervised and unsupervised methods. Papagiannopoulou and Tsoumakas (2020) gives a detailed overview towards existing works for keyphrases extraction. In the supervised setting, keyphrases extraction usually are treated as a classification problem (Alzaidy et al., 2019; Sahrawat et al., 2019) or text generation (Meng et al., 2017; Gu et al., 2016) needing large amounts of annotated data for training, the pipeline of which is different from ours. In this paper, we discuss unsupervised keyprhases extraction only. Traditional unsupervised methods are graph-based via converting a document into graphs, popular in early works. Motivated by Brin and Page (1998), TextRank (Mihalcea and Tarau, 2004) was proposed to rank nodes of graph constructed by word co-occurrence windows and implementing PageRank iteratively. After this, various works attempted to expand TextRank. SingleRank (Wan and Xiao, 2008) is one of modifications of TextRank that the weight of each edge is equal to the number of co-occurrences of two corresponding words. Florescu and Caragea (2017) argues that positions of a phrase appearance could be a vital heuristic for keyprhases extraction and they incorporated information from all positions of a word's occurrences into a biased PageRank. Wan and Xiao (2008) proposed to expand a specified document to a small document set by adding a few neighbor documents close to the document, and apply graph-based algorithm to make use of both the local information in the specified document and the global information in the neighbor documents. TopicRank (Bougouin et al., 2013) assigned a salience score to each topic by candidate keyphrase clustering. MultipartiteRank (Boudin, 2018) encodes topical information within a multipartite graph structure. In addtion, statistic-based methods are also one of previous popular works, the most effective of which is YAKE (Campos et al., 2018). YAKE incorperates five different features for each term to calculate a ranking score, after preprocessing the text by splitting it into individual terms.

Embedding-based methods rely on notable new developments in text representation learning by encoding text sequence into a low-dimension vector. Hence, in recent years, embedding-based unsupervised keyphrase extraction has gained a lot of attention in recent years. EmbedRank (Bennani-Smires et al., 2018) proposed to measure the text similarity between phrase and document embeddings to make prediction. SIFRank (Sun et al., 2020) proposed SIFRank, which improves the static embedding from EmbedRank with a pretrained language model and a sentence embedding model SIF (Arora et al., 2016). As we mentioned before, embedding-based model is severely restricted by the mismatch of sequence length for phrase and document. Subsequently, our method could address this issue by computing similarity between masked document and source document.

2.2 Contextual Embeddings

Contextual embeddings refer to representations for text data encoded by pre-trained language model, which is trained on large-scale unlabeled corpus to learn prior knowledge and then fine-tuned on downstream tasks. Compared with static word embedding, such as Word2Vec (Mikolov et al., 2013), GloVe (Pennington et al., 2014), and FastText (Bojanowski et al., 2017), contextual embeddings are entailed with rich syntactic and semantic knowledge. OOV problem could be alleviated via dynam-

ically encoding word and sentence embedding according to their context. In addition, contextual embeddings could provide more semantic information in contrast with Sen2Vec (Pagliardini et al., 2017) and Doc2Vec (Lau and Baldwin, 2016). ELMo (Peters et al., 2018), one of pre-trained model for contextual embeddings, concatenate the forward and backward layers of Bi-LSTM (Huang et al., 2015) model to capture bidirectional information, which ensure the fully utilization of entire context. Soon after, BERT (Devlin et al., 2018) is proposed and widely used in NLP area, the structure of which is bidirectional transformer that can capture better context information than ELMo. There are also a lot of other pre-trained language models such as RoBERTa (Liu et al., 2019), XLNET (Yang et al., 2019), etc. And for modeling longer text data, Beltagy et al. (2020); Zaheer et al. (2020) presented varients for BERT beneficial for long contexts by remedying quadratic dependency on the sequence length due to full attention mechanism. In this paper, we mainly use BERT to represent documents; meanwhile, we evaluate our methods on other pre-trained models to verify the utility of MDERank.

3 Problem Definition

Given a document $d = \{w_1, w_2, ..., w_n\}$ in dataset D, it has a candidate phrase set $C = \{c_1, c_2, ..., c_m\}$, where $m < n$. Each candidate c_x consist of several words $c_x = \{c_x^1, c_x^2, ..., c_x^l\}$. Keyphrase extraction is to select Top-K candidates from C, which are sorted according to their ranking scores. The selected candidates make up with a keyphrase list $K = \{k_1, k_2, ...k_t\}$, $t < m < n$.

4 Masked Document Embedding Rank

Since keyphrases are the most representative phrases giving concise summary towards the content, the semantics of a document would change a lot if we delete keyphrases from it. Based on this intuition, we propose to utilize a placeholder to imitate deletion and adopt cosine similarity to evaluate the difference of semantics. As a new unsupervised approach for keyphrase extraction task, MDERank automatically ranks candidates selected by linguistic rules without annotated data. Meanwhile, for a candidate, MDERank incorporates its frequency and offset position information implicitly by encoding the masked document into a sequence vector with BERT.

Compared with previous works like EmbedRank and SIFRank, our method is more robust on datasets in various domains and document length. This is owing to the context modeling capability of BERT, which dynamically represents tokens in different contexts, while the representation for each token is fixed in previous works. Different datasets have distinct preferences for keyphrases such like phrase length or phrase frequency. EmbedRank and SIFRank designed parameters well suitable for different datasets by introducing prior bias. Unlike previous works, we present four ways to mask a document with its candidate set, which could be chosen according to the characteristics of specific dataset.

4.1 Mask All

Mask All represents that if a candidate c_x appears multiple times in a document d , we would mask all the positions it appears. Therefore, the masked document d_m would be like $w_p = [MASK]$, p are the positions of a candidate in the document. This mask method may cause a bias for high frequency candidates because the more $[MASK]$, the more loss for the information as to a document.

4.2 Mask Once

Mask once only mask the first position of a candidate c_x in a document. $w_{p1} = [MASK]$, p_1 is the first position appearance of a candidate in a document. Mask once prefers longer candidates since longer candidates contains more semantics, sorting short candidates to the back. Mask once is uneven for keyphrases with less subwords but high frequency. If a dataset has a strong bias for long-length keyphrases, this method helps a lot.

4.3 Mask Highest

To balance frequency and length, for candidate c_x, we propose a third method: selecting the highest cosine similarity score of c_x in different positions $p_l, l \in N$ as the representative score for c_x used for the final ranking, which could be seem as a normalization on the two variables. For a candidate, its embeddings in different positions are varied from the context. Therefore, the variance between different embeddings for the same candidate could induce different masked document embeddings. Proposed that a candidate "network" has three appearances, randomly choosing one to join in the final ranking may lead to make the meaningless "network" to be considered, which is harmless to the global semantics. It is unfair

for "network" since it would lose competitiveness compared with other candidates who send the most competitive appearance to mask for the final ranking. That is why we force each candidate select the most "competitive" one to generate mask document and calculate the cosine similarity with their corresponding document.

4.4 Mask Subset

Diversity is a key indicator for evaluating the selected keyphrases. Hence, we propose to mask subset to improve the diversity of our method. Take candidate $c_x = \{w_1, w_2, ..., w_m\}$ for instance, if there is another candidate $c_y = \{w_1, w_2, ..., w_n\}, c_y \subset c_x$, the position of c_x, denoted as a set $p_{cx} = \{p_1, p_2, ...p_m\}$, and position set $p_{cy} = \{p_1, p_2, ...p_n\}$ for c_y, $p_x \cup p_y = p_x$. At the beginning, we sort all the candidates based on their subwords number in a descending order. Next, we calculate cosine similarity for each candidate in order. When mask document with c_y, we only mask the subtraction of positions set p_x and p_y for candidate c_y to generate d_{my}. In this way, the cosine score for overlapping candidates would widen the gap for the reason that the masked number for candidate c_y will decrease, yielding c_y with a lower ranking. Therefore, for nested candidates, Mask Subset could easily separate them and prefer a longer candidate. The detail of Mask Subset is shown in Algorithm 1.

4.5 Ranking Score Computation

After generating masked document d_m, d_m would be send to BERT base model. BERT model would encoded d_m into vectors $\mathbf{e_{dm}}$. Meanwhile, the source document without mask will do the same operation to obtain its embedding $\mathbf{e_{do}}$. Aimed to estimate the semantic similarity, we adopt cosine similarity as EmbedRank and SIFRank did. But BERT sentence embedding space is semantically non-smoothing and poorly defined in some areas, which makes it hard to be used directly through simple similarity metrics such as dot product or cosine similarity, so we decide introduce Euclidean distance as well. The functions are shown as below.

$$\cos(\mathbf{e_m}, \mathbf{e_o}) = \frac{\sum_{i=1}^{n} \mathbf{e_m^i} \mathbf{e_o^i}}{\sqrt{\sum_{i=1}^{n} \mathbf{e_m^i}} \sqrt{\sum_{i=1}^{n} \mathbf{e_o^i}}} \quad (1)$$

Algorithm 1 Mask Subset

Input: Document Set $D = \{d_i\}, i \in M$
Output: Masked Document Set

```
for d_i ∈ D do
    Masked Position set pos_i ← Φ.
    Generate Candidates set C_i = {c_i^j}, j ∈ N.
    Sort candidate set C_i by the number of subwords in a descending order.
    for c_j ∈ C_i do
        Get c_j Position Set p_c
        if p_c ∩ pos_i ≠ φ then
            Mask position p_c - pos_i ∪ p_c and generate masked document d_mj
        else
            Mask position p_c and generate masked document d_mj
        end if
    end for
end for
return Masked Document Set D_m = {d_mj}, j ∈ N
```

$$D(\mathbf{e_m}, \mathbf{e_o}) = \sqrt{\sum_{i=1}^{n} (\mathbf{e_m^i} - \mathbf{e_o^i})^2} \quad (2)$$

where n is the length of document. e_m and e_o are masked document and its source document embeddings seperately.

5 Document-Level Embedding

MDERank compares the embeddings of source document and masked document by aggregating token-level embeddings encoded by BERT. The BERT used for MDERank is pre-trained language model for the purposed to better represent documents with MASK placeholders.

5.1 Token Embedding

BERT model encodes document d into a sequence of vectors $\mathbf{E_d} = \{\mathbf{e_d^1}, \mathbf{e_d^2}, ..., \mathbf{e_d^n}\}$:

$$\mathbf{E_d} = BERT\{w_1, w_2, ..., w_n\} \quad (3)$$

where $\mathbf{e_d^i}$ is the contextualized embedding of w_i from a BERT layer.

5.2 Token Aggregation

To obtain document-level embeddings, we adopt different methods to represent a document, mainly MaxPooling, MeanPooling, and CLS placeholder.

The performance of MDERank with different fusions for token embeddings are almost identical, which could see from Section 8.4.

6 Keyphrase-based Self-supervised learning

Nowadays pre-trained BERT models are not specially set for KPE tasks. The linguistic and semantic knowledge learned from masked language model(MLM) and next sentence prediction (NSP) do not include the characteristic of keyphrases. That is why we consider to train a innovative method to fine-tune BERT model, named KPEBERT. To better suit for our MDERank method, KPEBERT also adopt the idea that when we mask a "keyphrase", the semantic of a document would change a lot by pushing documents masked with their keyphrases far away from source documents. In other words, the similarity score between source document and masked document is larger than a document masked with a "non-keyphrase". The former is what we called a positive example while the later is a negative example. And the pseduo labels indicating which candidates are keyphrases are generated by YAKE, which is staleb and in a high-speed among unsupervised keyphrase extraction methods. To prevent the model over-relying on the result of YAKE, different sampling strategies are proposed, which are absolute and relative sampling conforming to the "non-keyphrase" selection. Meanwhile, we employed a triplet loss to force the BERT model learning how to tell different between keyphrases and non-keyphrases with a setting margin m. We also kept the training task for mask languge model (MLM) to make sure KPEBERT would not forget previous linguitic knowledge. Figure 3 shows the backbone of KPEBERT training. The loss function is shown in Function 6.

$$L_{CL} = max\left(d\left(\mathbf{e_a}, \mathbf{e_n}\right) - d\left(\mathbf{e_a}, \mathbf{e_p}\right) + m, 0\right) \tag{4}$$

$$L_{MLM} = CE\left(\mathbf{w_m}, \mathbf{w_g}\right) \tag{5}$$

$$L = L_{CL} + L_{MLM} \tag{6}$$

where $\mathbf{e_a}$, $\mathbf{e_p}$, $\mathbf{e_n}$ represent the embedding of source document, document masked with a keyphrase and document masked with a non-keyphrase. m is margin, and function d is a Euclidean distance. $\mathbf{w_m}$ and $\mathbf{w_g}$ refer to masked token and its ground truth embeddings. With training, negative example will be closer to anchor while positive example will be away from anchor, conforming to MDERank proposal that a document masked with its keyphrase will lose its crutical information and dissimilar to source document in semantics.

6.1 Absolute Sampling

Similar to SimCSE (Gao et al., 2021) framework, we pass the input documents into pre-trained encoder three times, and then by applying unsupervised keyphrases extraction method like YAKE, we could generate "keyphrases" as pseudo labels. However, "non-keyphrases" refers to candidates selected by POS tags and stopwords at first but are not counted as keyphrases by YAKE. They are ranked lower than a threshold n. Inspired by MLM, we changed the mask sampling method into masking "keyphrases" of a document to generate positive example D_p for a document D. For negative random, we select a "non-keyphrase" and mask it in document D to get D_n.

6.2 Relative Sampling

Although absolute sampling achieved promising performance, the first step of sampling that utilizing unsupervised keyphrase extraction method to generate pseudo labels will affect the model training in some extend. For the purpose that avoiding fine-tuned model imitating the previous methods and make it more suitable for MDERank, we propose a fine-grained sampling strategy that "non-keyphrases" are no longer the ones ranking lower than a threshold k. We randomly choose two candidates from the Top-K results of YAKE, and the one with a higher ranking will be regarded as a "keyphrase" while the other is a "non-keyphrase". Therefore, we could obtain positive and negative examples according to the ranking of two candidates, which could be seen as a learning to rank method teaching the model to distinguish which candidate is more likely a "keyphrase" rather than using hard labels as absolute sampling.

7 Experiments

In this section, we introduce the benchmark datasets and baselines.

7.1 Datasets

The training data are the WikiText language modeling dataset with a collection of over 100 million tokens extracted from the set of verified Good

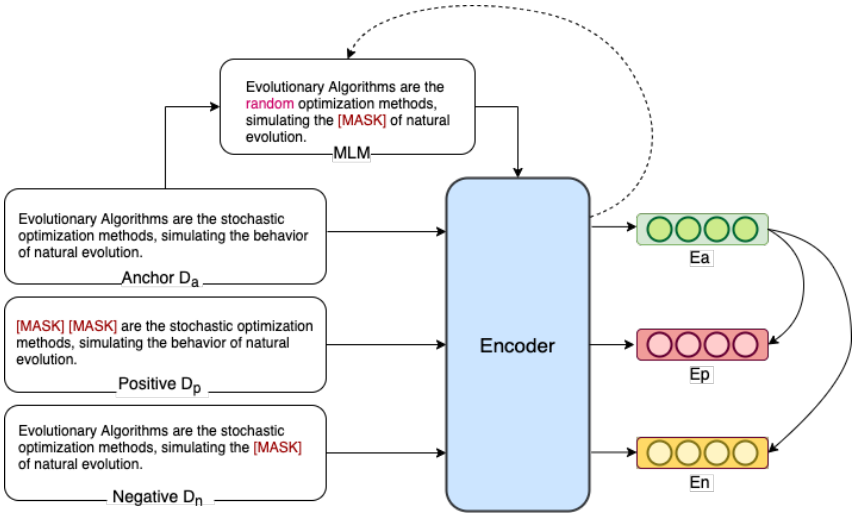

Figure 3: There are two task in the training process: one is teaching the encoder to distinguish documents masked with keyphrases and non-keyphrases. The other is fine tuning encoder with a MLM task. The word in pink is a case to illustrate the random replacement in MLM sampling.

Datasets	N_{KP}	L_{KP}	L_{Doc}
Inspec	9.82	2.31	121.84
SemEval2010	15.07	2.11	189.90
SemEval2017	17.30	3.00	170.38
DUC2001	8.08	2.07	724.63
NUS	11.66	2.07	7702
Krapivin	5.74	2.03	8544.57

Table 1: This table demonstrates the statistics of different datasets. N_{KP} represents the average number of gold keyphrases. L_{KP} is the average length of gold keyphrases. L_{Doc} is the average number of words for documents in individual dataset.

and Featured articles on Wikipedia[1]. There are six datasets used for evaluation and four of them are widely-adopted scientific publications[2], namely Inspec (Hulth, 2003), Krapivin (Krapivin et al., 2009), NUS (Nguyen and Kan, 2007), and SemEval-2010 (Kim et al., 2010), that have been widely used in existing literature (Meng et al., 2017; Chen et al., 2019; Sahrawat et al., 2019; Bennani-Smires et al., 2018; Meng et al., 2020). Plus, we also used DUC2001 dataset (Wan and Xiao, 2008) and SemEval2017 (Augenstein et al., 2017)[3] collected from news articles and scienceDirect journals, individually. The detailed description of datasets are presented in the supplementary document due to the page limitation. These datasets could be categorized into three types depending on the number of words: short documents,medium-length documents and long documents datasets. Inspec, SemEval2017, SemEval2010 belong to short documents datasets. (Note that we only integrate the abstract of documents in SemEval-2010.) DUC2001 is a medium-length datasets. Likewise, Krapivin and NUS are datasets with long documents. Table 1 provides some general statistics on these dataset used in this paper.

We follow the common practice and evaluate the performance of our model in terms of F-measure (F1) at the top N keyphrases (N is 5, 10, 15 in this paper), and apply stemming to reduce the number of mismatches. Meanwhile, the final predictions have been deduplicated for a fair comparison.

7.2 Baselines and Parameter Settings

We first compare our work with six unsupervised algorithms with a a widely used toolkit for keyphrase extraction: PKE[4], including TextRank, SingleRank, TopicRank, MultipartiteRank, YAKE and EmbedRank algorithms. We used the parameters default in PKE. As for SIFRank, we use the parameters suggested by the authors. All models are using the same tools for tokenizing, part-of-speech tagging and noun phrase chunking under the same environment. We use the Stanford CoreNLP[5]to tokenize and part-of-speech tag. And regular ex-

[1] https://huggingface.co/datasets/wikitext
[2] https://github.com/memray/OpenNMT-kpg-release
[3] https://github.com/sunyilgdx/SIFRank/tree/master/data
[4] https://github.com/boudinfl/pke
[5] https://stanfordnlp.github.io/CoreNLP/

Method	Inspec			SemEval2017			SemEval2010			DUC2001			Krapivin			NUS		
	F1 @5	F1@10	F1@15	F1 @5	F1@10	F1@15	F1 @5	F1@10	F1@15	F1 @5	F1@10	F1@15	F1 @5	F1@10	F1@15	F1 @5	F1@10	F1@15
TextRank	21.58	27.53	27.62	16.43	25.83	30.50	7.42	11.27	13.47	11.02	17.45	18.84	6.04	9.43	9.95	1.80	3.02	3.53
SingleRank	14.88	21.50	24.13	18.23	27.73	31.73	8.69	12.94	14.4	19.14	23.86	23.43	8.12	10.53	10.42	2.98	4.51	4.92
TopicRank	12.20	17.24	19.33	17.10	22.62	24.87	9.93	12.52	12.26	19.97	21.73	20.97	8.94	9.01	8.30	4.54	7.93	9.37
MultipartiteRank	13.41	18.18	20.52	17.39	23.73	26.87	10.13	12.91	13.24	21.70	24.10	23.62	9.29	9.35	9.16	6.17	8.57	10.82
YAKE	8.02	11.47	13.65	11.84	18.14	20.55	6.82	11.01	12.55	11.99	14.18	14.28	8.09	9.35	9.12	7.85	11.05	13.09
EmbedRank	14.51	21.02	23.79	20.21	29.59	33.94	9.63	13.9	14.79	21.75	25.09	24.68	8.44	10.47	10.17	2.13	2.94	3.56
SIFRank	**29.38**	**39.12**	**39.82**	**22.38**	**32.60**	**37.25**	11.16	16.03	18.42	**24.30**	**27.60**	**27.96**	1.62	2.52	3.00	3.01	5.34	5.86
Phrase-Document	28.92	**38.55**	**39.77**	20.03	31.01	36.72	10.46	16.35	19.35	8.12	11.62	13.58	4.05	6.60	7.84	3.75	6.34	8.11
MDERank$_{once}$	27.93	37.38	39.11	20.56	30.95	36.07	10.16	15.40	17.69	9.11	13.49	16.47	4.61	7.21	8.15	3.92	6.52	8.85
MDERank$_{highest}$	27.93	37.42	39.36	20.56	30.97	36.10	10.16	15.32	17.76	9.11	13.46	16.45	4.65	7.24	8.20	3.92	6.56	8.85
MDERank$_{subset}$	**29.25**	36.55	38.08	21.50	31.30	36.67	10.26	15.88	17.83	12.05	16.73	19.19	8.50	9.99	10.48	9.61	13.43	14.65
MDERank$_{all}$	26.17	33.81	36.17	**22.81**	**32.51**	37.18	12.95	17.07	20.09	13.05	17.31	19.13	11.78	12.93	12.58	**15.24**	**18.33**	**17.95**
MDERank + KPEBERT$_{ab}$	28.06	35.80	37.43	21.63	32.23	**37.52**	**12.95**	**17.95**	**20.69**	22.51	**26.97**	26.28	**12.91**	**14.36**	**13.58**	14.11	17.72	**17.95**
MDERank + KPEBERT$_{re}$	27.85	34.36	36.40	20.37	31.21	36.63	**13.05**	**18.27**	**20.35**	**23.31**	26.65	**26.42**	**12.35**	**14.31**	**13.31**	**14.39**	**18.46**	**19.41**

Table 2: The best performance for $F1@5$, $F1@10$, $F1@15$ are underlined and bold. We bolded the second-performing result. Ab and Re are absolute and relative sampling respectively. The method for MDERank+KPEBERT is Mask All for both absolute and relative sampling strategies. Phrase-Document is the pipeline used by previous embedding-based methods but we use BERT to encode documents and phrases.

pression <*NN.* * |*JJ*> * <*NN.**> is used to extract noun phrases as the candidate keyphrases

The number of pseudo labels for absolute and relative KPEBERT training are set as 10 and 20. The optimizer setting we toke are as same as the ones in SimCSE framework (Gao et al., 2021). The margin of triplet loss is 0.2. We used 4 V100 for distributed training and the batch size is 2 for per device and the gradient accumulation is 4. As for learning rate, we used 3e-5 for default.

8 Results and Analysis

In this section, we report the main observations for the different experiments that we performed. The first experiment demonstrates the superiority of MDERank and its variants by comparing with the unsupervised baseline models and state-of-the-art method SIFRank in terms of several metrics. We also prove that KPEBERT could improve the performance of MDERank in some extend. The second one is to analyze the effect of sequence length for MDERank to verify that longer document could benefit from MDERank. The third one is to prove that Mask Subset can improve the diversity of ranking results. The fouth one is to analyze the sensitivity of the different layers and distance metrics for phrase-document based models compared with MDERank. We also analyze the sensitivity of the hyperparameters and ablation study on MLM task and heuristic keyphrases extraction methods in the supplementary.

8.1 Performance Comparison

Regardless of the presence or absence of keyphrases, we compare the capability of model prediction for whole keyphrases, which is more reasonable for real applications. We show the F1 scores at the top 5, 10 and 15 predictions on Table 2. It could be seen from Table 2 that MDERank and its variants perform stably on six benchmarks and exceed SIFRank and EmbedRank on most datasets, especially on long document datasets NUS and Krapivin. For Inspec and DUC2001, we could find that MDERank and it variants are nearly 1% worse than SIFRank method on different F1 metrics. Before we run variants and KPEBERT, MDERank did not perform well on these two datasets with a big gap with SOTA on Inspec and DUC2001. However, after employing variants with strong bias — reducing the number of MASK, the gap between our method and SOTA work is narrowed on Inspec. Therefore, we could get results fairly close to SIFRank. Besides, if we move candidates with only 1 token to the end of ranking, MDERank$_{all}$ gained 29.71% ,38.15%, 39.46% for F1@5, F1@10, F1@15 individually, increasing 3.54%, 4.34% and 3.29% compared with unchanged MDERank$_{all}$. We could infer from these that Inspec prefers keyphrases with more tokens. Additionally, considering the average sequence length of gold labels in Inspec, phrase-document based methods could easily gain from this strong bias. This is because that they highly hinge on the sequence length since the longer the phrase, the better quality of embedding and the accurate the computation of similarity between candidate and its document is. In this way, we can explain why MDERank$_{all}$ performs poorer than SIFRank and phrase-document on Inspec.

Unlike other datasets collected from scientific publications, DUC2001 are news articles which is in open-domain and the writing style dissimilar with science papers. Although MDERank$_{all}$ works unpleasant on DUC2001, it can be signif-

	NUS (512)			**NUS** (2000)			**Krapivin** (512)			**Krapivin** (2500)		
	F1 @5	F1@10	F1@15	F1 @5	F1@10	F1@15	F1 @5	F1@10	F1@15	F1 @5	F1@10	F1@15
MDERank	15.24	18.33	17.95	–	–	–	11.78	12.93	12.58	–	–	–
MDERank+BigBird	15.42	17.68	17.81	15.36	19.56	20.33	11.62	11.99	11.70	11.33	12.71	12.70

Table 3: 512, 2000, 2500 represent words number for each document in datasets. The results for NUS(2000) and Krapivin (2500) are missing due to the limitation of BERT-based models for sequence length.

Method	Word Num	**F1@5**	**F1@10**	**F1@15**
Phrase-Document	128	8.76	14.75	16.28
	256	5.86	10.19	12.90
	512	3.75	6.34	8.11
MDERank	128	12.86	16.06	16.67
	256	14.45	16.01	16.64
	512	15.24	18.33	17.95

Table 4: This table illustrates words number leads to the change of F1@K score for phrase-document and MDERank on NUS dataset.

icantly improved by replacing BERT-base model with KPEBERT, which are trained on open-domain dataset.

Especially, MDERank outperformed baseline methods on long document datasets. To better explain this result, we discuss how the length of a document has effect on MDERank in Section 8.2. Absolute and relative sampling for KPEBERT achieved nearly similar performance on 6 datasets, while relative sampling exceeds absolute sampling on NUS remarkably.

8.2 Impact of Sequence Length

As described in Section 8.1, MDERank has achieved convincing results on long documents datasets. The prominent improvement results from the fact that MDERank ensures the match of sequence length as we compute the text similarity for ranking. In order to support this view, we conduct two experiments. The first one explored the impact of different input length on NUS. At beginning, we truncated a document within 128, 256 and 512 words. In Table 4 , the quality of keyphrases predicted by phrase-document sharply decreases as the words number increases, but MDERank performs stable and even has improvement along with the increase of words number. This reflects phrase-document based methods are substantially affected by sequence length. However, MDERank advances in the increase of sequence length, showing a more powerful and potential capability of keyphrase extraction for longer document.

The second experiment is to figure out whether MDERank performance for longer document will be improve if we replace BERT with BigBird, which has the capability to handle longer context. Despite that we only adopt documents with 512 words for BERT model by truncation, this setting is not the best choice for different datasets. Firstly, we made statistics on DUC2001, NUS and Krapivin to discover the optimal sequence length to cover most of gold phrases for individual dataset. From Figure 4, 500 words nearly covers 90% gold keyphrases in DUC2001, half of gold keyphrases in Krapivin are concentrated in the first 2500 words, while NUS are centralized in the first 2000 words. Moreover, Figure 4 tells that the upper boundaries for DUC2001, NUS and Krapivin are around 97%, 70% and 66%, impossible for any methods realizing an accuracy at 100%. Considering we have truncated the documents to 512 words already and the vertex of performance for DUC2001 is approximate 500 words, we decide not to do experiments on this dataset. Instead, we choose the first 2500 and 2000 words for documents from Krapivin and NUS as input documents to further test the impact of sequence length. The encoder for MDERank is replaced by Big Bird (Zaheer et al., 2020) in order to encode longer sequence better. Table 3 demonstrates that increasing the words number for input documents can favor the performance of MDERank. This is because more context and candidates would be involved into the final ranking. Otherwise, truncation will ignores some candidates which are gold automatically, intruding some keyphrases to be retrieved successfully.

8.3 Diversity

In order to enhance the diversity of extracted keyphrases, we proposed Mask Subset for MDERank, which a slot can only be masked once. Therefore, the nested phrases which contains more than one candidates, are allowed to participate into the ranking only once. For example, candidate "network" are part of candidate "neural network". If the latter candidate is picked and masked firstly, the

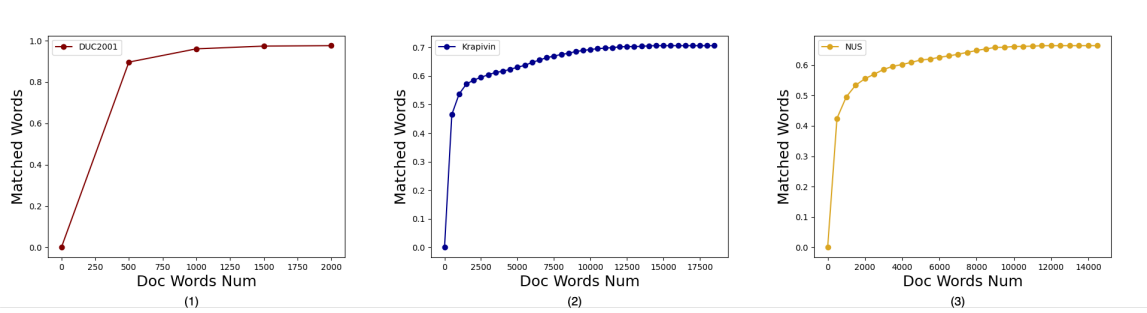

Figure 4: The example document is underlined and the orange line represents a overlapping keyphrase. The results for baseline matched with gold labels are marked in red, while the results for MDERank are marked in green. The y-axis represents the average portion of words in gold keyphrases matched in first m words, where m is the number of words in documents.

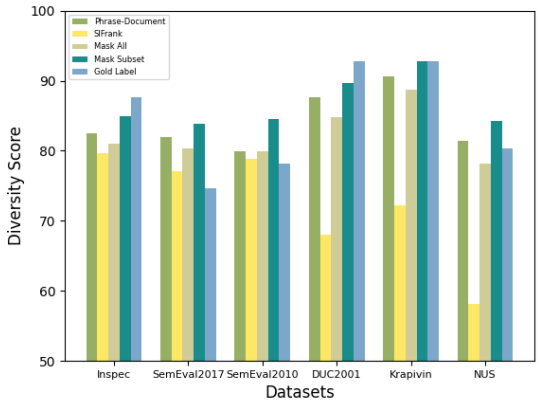

Figure 5: This figure shows the diversity score for different methods on various datasets. A lower bar means a better diversity. The diversity of gold keyphrases are in blue and on the right.

Pooling	Method	Layer	DUC2001		
			F1@5	F1@10	F1@15
AvgPooling	Phrase-Document	3	16.19	21.21	22.12
		6	10.76	15.33	17.63
		12	10.41	15.15	17.69
	MDERank	3	12.00	16.45	19.08
		6	12.40	17.07	19.02
		12	13.00	17.93	19.45
MaxPooling	Phrase-Document	3	6.97	11.04	12.27
		6	7.12	10.93	13.13
		12	8.12	11.62	13.58
	MDERank	3	11.06	16.16	18.01
		6	11.06	15.91	17.98
		12	13.05	17.32	19.14

Table 5: This table exhibits the performance of different layers and pooling methods for MDERank on DUC2001 dataset. AvgPooling and MaxPooling are employed on the output of specific layer to produce document-level embeddings. Here, we adopted Mask All for MDERank.

former candidate would not be considered even if it is selected afterwards. Mask Subset could prevent repeatedly masking synonyms in some extend. The metric for diversity evaluation is:

$$Diveristy(d) = \frac{t_u}{t_n} * 100 \quad (7)$$

Where t_u and t_n are the number of unique tokens and total number of tokens in predicted keyprhases respectively. The higher metric means a better diversity.

Figure 5 shows the diversity of keyphrases predicted by Phrase-Document, SIFRank, Mask All and Mask Subset. Jointly, we compute the diversity of gold phrases. Among the four extaction methods, Mask Subset achieved the highest score on 6 datasets, and surpasses gold keyphrases on half of datasets;meanwhile, the diversity on Krapivin is almost the same with ground truth. The experiment results prove the effectiveness of Mask Subset, which not only has high diversity but a competitive performance on predictions according to Table 2.

8.4 Impact of Layers and Pooling Ways

Jawahar et al. (2019); Kim et al. (2020); Rogers et al. (2020) mentioned that BERT captures a rich hierarchy of linguistic information, with surface features in lower layers, syntactic features in middle layers and semantic features in higher layers. In this paper, we picked the third layer as surface layer, the sixth layer as syntactic layer and the last layer as semantic layer. As we can see from Table 5, with AvgPooling, Phrase-Document drastically plunges its F1 score as the layer rises from 3 to 12. On the contrary, MDERank remains steady as to the change of layers. Compared with AvgPooling, MaxPooling gives weaker representations for documents and severely lowers the performance of Phrase-Document method. Even though Phrase-

Document is stable to the factor of layers with MaxPooling representation, the results are unreliable considering the poor document embeddings. Still, the performance of MDERank with MaxPooling is similar to it with AvgPooling, which shows little change to layer changes. For MDERank, the last layer achieved the best result no matter using AvgPooling or MaxPooling. This is because the length of input sequence for MDERank is the same, allowing two embeddings space aligned. As a result, when computing the similarity of two matched representation spaces, stronger representations will show a prominent performance.

9 Conclusion

In this work, we propose MDERank, a simple and efficient unsupervised keyphrase extraction method, which improves state-of-the-art unsupervised-embedding-based methods for keyphrases extraction tasks. Moreover, we take the bias of different datasets into consideration that we designed three other variants for MDERank. To improve diversity of predictions, we bring up with Mask Subset, an algorithm with high diversity and competitive results on F1 score. We present an self-supervised approach for BERT training which enhanced the representation of document embeddings predicts. We further justify the inner workings of our approach by analyzing the impact of sequence length for MDERank along with other baseline models. Furthermore, we explain the essential reason for the superiority of MDERank compared with baseline models. In addition, we verified MDERank is more robust than previous methods via investigating how the way to get representation of document embeddings influences on the performance of embedding-based methods and MDERank. In conclusion, MDERank is an efficient and robust unsupervised method for keyphrases extraction task, which deals with the issue among in previous work that the representation of text to calculate similarity is mismatched owing to the unequal sequence length of inputs.

Acknowledgements

This work was supported by Alibaba Group through Alibaba Research Intern Program.

References

Rabah Alzaidy, Cornelia Caragea, and C Lee Giles. 2019. Bi-lstm-crf sequence labeling for keyphrase extraction from scholarly documents. In *The world wide web conference*, pages 2551–2557.

Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2016. A simple but tough-to-beat baseline for sentence embeddings.

Isabelle Augenstein, Mrinal Das, Sebastian Riedel, Lakshmi Vikraman, and Andrew McCallum. 2017. Semeval 2017 task 10: Scienceie-extracting keyphrases and relations from scientific publications. *arXiv preprint arXiv:1704.02853*.

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Kamil Bennani-Smires, Claudiu Musat, Andreea Hossmann, Michael Baeriswyl, and Martin Jaggi. 2018. Simple unsupervised keyphrase extraction using sentence embeddings. *arXiv preprint arXiv:1801.04470*.

Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146.

Florian Boudin. 2018. Unsupervised keyphrase extraction with multipartite graphs. *arXiv preprint arXiv:1803.08721*.

Adrien Bougouin, Florian Boudin, and Béatrice Daille. 2013. Topicrank: Graph-based topic ranking for keyphrase extraction. In *International joint conference on natural language processing (IJCNLP)*, pages 543–551.

Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual web search engine. *Computer networks and ISDN systems*, 30(1-7):107–117.

Ricardo Campos, Vítor Mangaravite, Arian Pasquali, Alípio Mário Jorge, Célia Nunes, and Adam Jatowt. 2018. Yake! collection-independent automatic keyword extractor. In *European Conference on Information Retrieval*, pages 806–810. Springer.

Wang Chen, Hou Pong Chan, Piji Li, Lidong Bing, and Irwin King. 2019. An integrated approach for keyphrase generation via exploring the power of retrieval and extraction. *arXiv preprint arXiv:1904.03454*.

Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. 2020. Electra: Pre-training text encoders as discriminators rather than generators. *arXiv preprint arXiv:2003.10555*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Corina Florescu and Cornelia Caragea. 2017. Positionrank: An unsupervised approach to keyphrase extraction from scholarly documents. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1105–1115.

Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. Simcse: Simple contrastive learning of sentence embeddings. *arXiv preprint arXiv:2104.08821*.

Jiatao Gu, Zhengdong Lu, Hang Li, and Victor OK Li. 2016. Incorporating copying mechanism in sequence-to-sequence learning. *arXiv preprint arXiv:1603.06393*.

Zhiheng Huang, Wei Xu, and Kai Yu. 2015. Bidirectional lstm-crf models for sequence tagging. *arXiv preprint arXiv:1508.01991*.

Anette Hulth. 2003. Improved automatic keyword extraction given more linguistic knowledge. In *Proceedings of the 2003 conference on Empirical methods in natural language processing*, pages 216–223.

Ganesh Jawahar, Benoît Sagot, and Djamé Seddah. 2019. What does bert learn about the structure of language? In *ACL 2019-57th Annual Meeting of the Association for Computational Linguistics*.

Chen Jia, Yuefeng Shi, Qinrong Yang, and Yue Zhang. 2020. Entity enhanced bert pre-training for chinese ner. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6384–6396.

Su Nam Kim, Olena Medelyan, Min-Yen Kan, and Timothy Baldwin. 2010. Semeval-2010 task 5: Automatic keyphrase extraction from scientific articles. In *Proceedings of the 5th International Workshop on Semantic Evaluation*, pages 21–26.

Taeuk Kim, Jihun Choi, Daniel Edmiston, and Sanggoo Lee. 2020. Are pre-trained language models aware of phrases? simple but strong baselines for grammar induction. *arXiv preprint arXiv:2002.00737*.

Mikalai Krapivin, Aliaksandr Autaeu, and Maurizio Marchese. 2009. Large dataset for keyphrases extraction.

Jey Han Lau and Timothy Baldwin. 2016. An empirical evaluation of doc2vec with practical insights into document embedding generation. *arXiv preprint arXiv:1607.05368*.

Yang Liu and Mirella Lapata. 2019. Text summarization with pretrained encoders. *arXiv preprint arXiv:1908.08345*.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Rui Meng, Xingdi Yuan, Tong Wang, Sanqiang Zhao, Adam Trischler, and Daqing He. 2020. An empirical study on neural keyphrase generation. *arXiv preprint arXiv:2009.10229*.

Rui Meng, Sanqiang Zhao, Shuguang Han, Daqing He, Peter Brusilovsky, and Yu Chi. 2017. Deep keyphrase generation. *arXiv preprint arXiv:1704.06879*.

Rada Mihalcea and Paul Tarau. 2004. Textrank: Bringing order into text. In *Proceedings of the 2004 conference on empirical methods in natural language processing*, pages 404–411.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Thuy Dung Nguyen and Min-Yen Kan. 2007. Keyphrase extraction in scientific publications. In *International conference on Asian digital libraries*, pages 317–326. Springer.

Matteo Pagliardini, Prakhar Gupta, and Martin Jaggi. 2017. Unsupervised learning of sentence embeddings using compositional n-gram features. *arXiv preprint arXiv:1703.02507*.

Eirini Papagiannopoulou and Grigorios Tsoumakas. 2020. A review of keyphrase extraction. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 10(2):e1339.

Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543.

Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. Deep contextualized word representations. *arXiv preprint arXiv:1802.05365*.

Anna Rogers, Olga Kovaleva, and Anna Rumshisky. 2020. A primer in bertology: What we know about how bert works. *Transactions of the Association for Computational Linguistics*, 8:842–866.

Dhruva Sahrawat, Debanjan Mahata, Mayank Kulkarni, Haimin Zhang, Rakesh Gosangi, Amanda Stent, Agniv Sharma, Yaman Kumar, Rajiv Ratn Shah, and Roger Zimmermann. 2019. Keyphrase extraction from scholarly articles as sequence labeling using contextualized embeddings. *arXiv preprint arXiv:1910.08840*.

Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. 2019. How to fine-tune bert for text classification? In *China National Conference on Chinese Computational Linguistics*, pages 194–206. Springer.

Yi Sun, Hangping Qiu, Yu Zheng, Zhongwei Wang, and Chaoran Zhang. 2020. Sifrank: a new baseline for unsupervised keyphrase extraction based on pre-trained language model. *IEEE Access*, 8:10896–10906.

Xiaojun Wan and Jianguo Xiao. 2008. Single document keyphrase extraction using neighborhood knowledge. In *AAAI*, volume 8, pages 855–860.

Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. 2019. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in neural information processing systems*, 32.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. 2020. Big bird: Transformers for longer sequences. In *NeurIPS*.